

Enterprise AI Prediction Workflow

AI as a Service, AutoML, LLM-Based Service, and API Integration

Project Overview

This report documents the creation of a local AI-as-a-Service workflow for demand prediction using sales data exported from Snowflake. The objective was to build a demand model with Quantity as the target variable, deploy the model as a service, compare the standard/manual model-building approach with AutoML, and then expose the service through an LLM-based natural-language interface.

Data, Variables, and Transformations

The dataset used for the assignment was exported from Snowflake as sales_data.csv. The model target was QUANTITY, and the selected explanatory variables were MATERIAL_DESCRIPTION, SIM_DATE, and NET_PRICE. These variables were chosen because product identity captures item-level demand differences, the date field captures timing and seasonality, and price is a reasonable business driver of demand.

Several transformations were necessary before model training. First, the original SIM_DATE field only contained month/day values, so a reference year was appended and the field was converted to a valid datetime type. Month and year were then extracted to create time-based explanatory variables. For the manual YDF model, MATERIAL_DESCRIPTION was label-encoded into PRODUCT_ID because the manual pipeline expected numeric input. NET_PRICE and QUANTITY were explicitly cast to numeric values. These steps ensured that the dataset could be used consistently in both the manual model and the AutoML workflow.

Selected Modeling Configuration

- Target variable: QUANTITY
- Manual model (pre-selected algorithm): YDF Gradient Boosted Trees
- Manual-model features: PRODUCT_ID, MONTH, YEAR, NET_PRICE
- AutoML approach: H2O AutoML using MATERIAL_DESCRIPTION, MONTH, YEAR, and NET_PRICE
- Services deployed locally with FastAPI and tested in Postman

a) Output Screenshots for Different AI Models and Services

The screenshots below document the outputs requested in the assignment: the manual/pre-selected model, the deployed manual API, the AutoML leaderboard and deployed AutoML API, and the LLM-based service tested with different prompts in Postman.

```
# Cell 11
predictions = model.predict(test_df[features])

rmse = mean_squared_error(test_df[target], predictions) ** 0.5
r2 = r2_score(test_df[target], predictions)

print(f"RMSE: {rmse:.2f}")
print(f"R2 Score: {r2:.4f}")

... RMSE: 1687.18
R2 Score: 0.4371
```

Figure 1. Manual YDF demand model evaluation output. The manual model produced RMSE = 1687.18 and $R^2 = 0.4371$ on the holdout test set

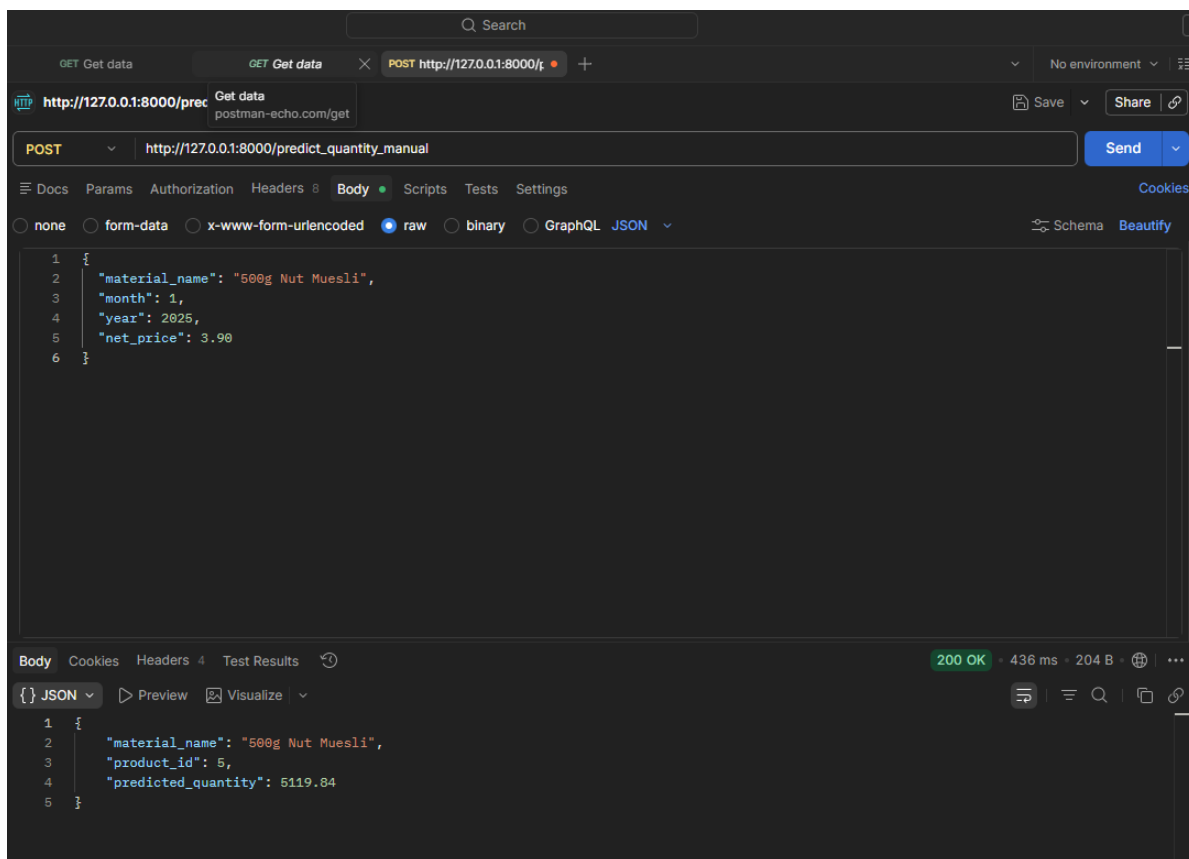


Figure 2. Manual YDF API tested in Postman. The service accepts material name, month, year, and net price, and returns a predicted quantity

```
# Cell 11
print(aml.leaderboard)
```

model_id	rmse	mse	mae	rmsle	mean_residual_deviance
XRT_1_AutoML_1_20260410_95145	1483.76	2.20154e+06	1029.33	0.793492	2.20154e+06
GBM_5_AutoML_1_20260410_95145	1483.77	2.20157e+06	1032.51	0.794973	2.20157e+06
GBM_4_AutoML_1_20260410_95145	1484.87	2.20484e+06	1027.36	0.785418	2.20484e+06
GBM_2_AutoML_1_20260410_95145	1486.8	2.21056e+06	1018.58	0.78814	2.21056e+06
GBM_3_AutoML_1_20260410_95145	1492.82	2.2285e+06	1021.46	0.785981	2.2285e+06
DRF_1_AutoML_1_20260410_95145	1500.74	2.25223e+06	1056.13	0.795466	2.25223e+06
StackedEnsemble_BestOfFamily_1_AutoML_1_20260410_95145	1519.01	2.3074e+06	1063.19	0.79524	2.3074e+06
XGBoost_2_AutoML_1_20260410_95145	1539.46	2.36992e+06	1098.78	0.797512	2.36992e+06
StackedEnsemble_AllModels_1_AutoML_1_20260410_95145	1565.88	2.45199e+06	1089.69	0.798184	2.45199e+06
XGBoost_3_AutoML_1_20260410_95145	1571.06	2.46824e+06	1108.65	0.80576	2.46824e+06

[12 rows x 6 columns]

Figure 3. H2O AutoML leaderboard. The top-ranked model was XRT_1_AutoML_1_20260410_95145 with the lowest RMSE among the tested models

The screenshot shows a Postman interface for testing an API. The request is a POST to `http://127.0.0.1:3000/predict_quantity_automl`. The body is raw JSON:

```
{
  "material_name": "500g Nut Muesli",
  "month": 1,
  "year": 2025,
  "net_price": 3.90
}
```

The response is a 200 OK with a status of 1.18 s and 189 B. The response body is JSON:

```
{
  "material_name": "500g Nut Muesli",
  "predicted_quantity": 5075.14
}
```

Figure 4. AutoML API tested in Postman. The deployed AutoML service returned a predicted quantity of 5075.14 for the sample request

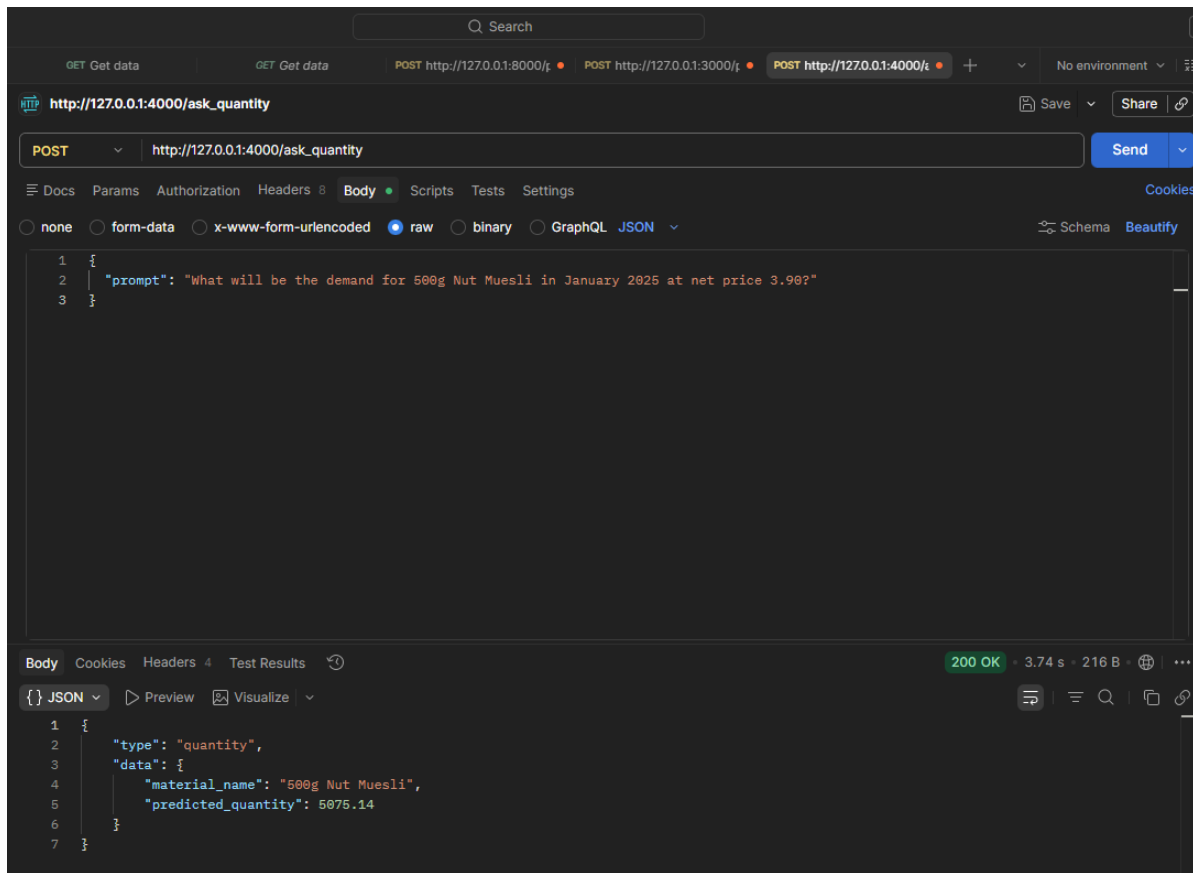


Figure 5. LLM router with a valid quantity prompt. The router classified the request as a quantity query, extracted the needed fields, and routed the request to the AutoML quantity service

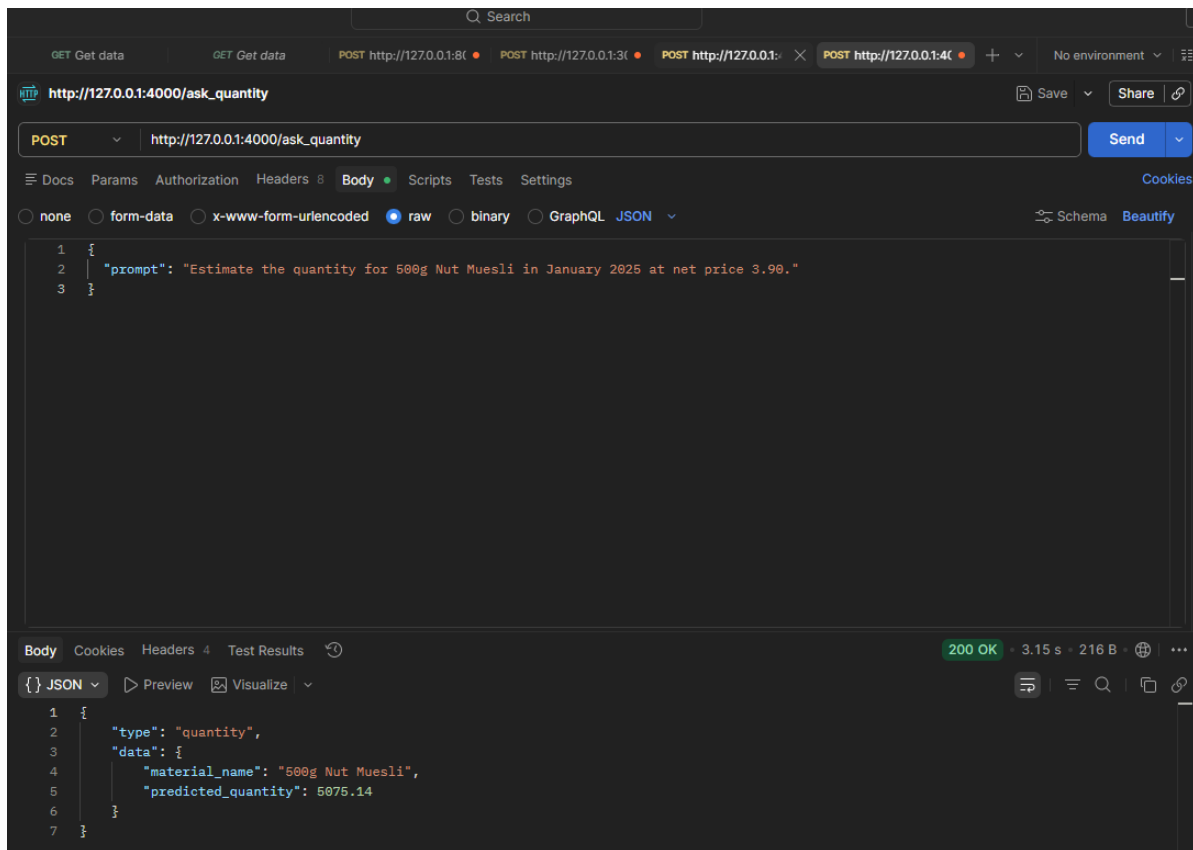


Figure 6. LLM router with an alternate valid phrasing. The service successfully interpreted a differently worded demand question and returned the same quantity prediction

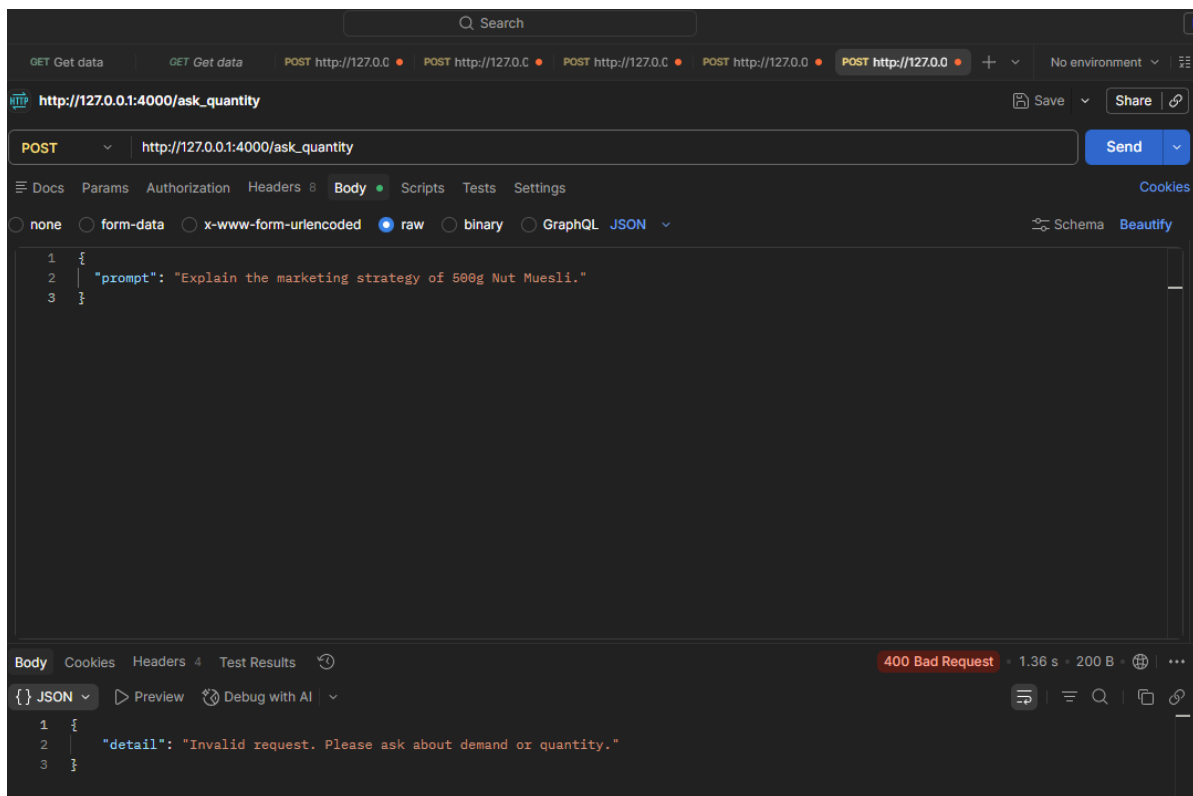


Figure 7. LLM router with an invalid/unsupported prompt. The service returned a 400 Bad Request message, showing that unsupported requests are rejected instead of being hallucinated

b) How Model Creation, Testing, and Validation Differ in AutoML vs. the Standard Approach

The standard/manual approach required me to explicitly choose the modeling algorithm, define the preprocessing steps, select the feature set, encode the categorical product variable, split the data manually, train the model, and then evaluate the model using RMSE and R^2 . In other words, the model-building logic was designed step by step by the user. This makes the workflow transparent and easy to explain, but it also places more responsibility on the user to make the right design choices.

In contrast, the AutoML workflow used H2O AutoML to automate much of the experimentation process. After the cleaned dataset and target were provided, AutoML tested multiple candidate models and returned a ranked leaderboard. This reduced the burden of manually choosing a single modeling algorithm. It also made comparison easier because model quality was summarized in one place rather than requiring separate manual training cycles for each algorithm.

Testing and validation also differed. In the manual workflow, I evaluated one selected model on a train/test split and interpreted the resulting RMSE and R^2 directly. In the AutoML workflow, H2O automatically generated leaderboard metrics and cross-validation-based comparisons across multiple models. This makes validation broader, because the user is not relying on only one manually selected model. However, it is also slightly less transparent because AutoML makes several internal decisions automatically, such as dropping constant or unhelpful variables.

Overall, the manual approach is helpful when the analyst wants direct control over algorithm choice, feature engineering, and interpretation, while AutoML is helpful when the goal is to efficiently search for the best-performing model with less manual experimentation. In this project, the AutoML approach produced a lower RMSE than the manual YDF model, which suggests stronger predictive performance for the final deployed quantity service.

c) Value of Deploying the Demand Prediction Model as a Service and Additional Steps Needed for Operational Use

Deploying the demand model as a service creates business value because prediction is no longer trapped inside a notebook. Once the model is exposed through an API, other systems or users can request demand predictions on demand using consistent logic. This improves scalability, repeatability, and integration potential. Instead of retraining or manually running code every time a planner needs a forecast, the same service can be called repeatedly from dashboards, workflow tools, planning systems, or decision-support applications.

From an AI and enterprise integration perspective, the service-based design also separates responsibilities more clearly. The model becomes one reusable component, while interfaces such as Postman, a front-end tool, or an LLM router can sit above it. This modularity is useful because a business can update the model, change the front end, or replace one service without rewriting the whole workflow.

To operationalize the demand service for continuous decision-making and MRP use, additional steps would be needed beyond the current prototype. First, the data extraction from Snowflake should be automated on a schedule so the model is trained or refreshed with up-to-date sales data. Second, a defined retraining policy is needed, such as weekly or monthly retraining depending on volatility in demand and pricing. Third, model monitoring should be put in place to track drift, forecast error, API uptime, and unusual inputs. Fourth, access, logging, and version control are needed so planners know which model version generated a forecast and so the service can be audited if decisions are challenged later.

For actual MRP use, the service would also need to be connected to a planning process rather than used as a standalone tool. That means the demand forecast should feed into replenishment or planning logic, trigger review thresholds when forecast values change materially, and be supported by business rules for exceptions, approvals, and overrides. In practice, a production version would include scheduling, monitoring, governance, and integration into downstream planning workflows before it could be treated as a reliable enterprise service.

d) Different LLM Prompts, Objective of Each Prompt, Output Generated, Value of the LLM-Based AI Service, and Challenges

The LLM-based service was created as a natural-language interface on top of the deployed quantity service. Instead of making the user format a raw API request, the LLM router accepts a plain-English prompt, identifies whether the request is about demand/quantity, extracts the relevant fields, and then calls the underlying AutoML demand API.

Prompt Exploration Summary

Prompt	Objective	Output Generated	What It Demonstrates
“What will be the demand for 500g Nut Muesli in January 2025 at net price 3.90?”	Test a direct demand-prediction prompt.	The service classified the prompt as a quantity request and returned predicted_quantity = 5075.14.	Shows successful end-to-end routing from natural language to the AutoML demand service.
“Estimate the quantity for 500g Nut Muesli in January 2025 at net price 3.90.”	Test whether the interface can understand an alternate phrasing.	The service again classified the request as a quantity prompt and returned predicted_quantity = 5075.14.	Shows robustness to prompt wording variation rather than dependence on one fixed phrasing.
“Explain the marketing strategy of 500g Nut Muesli.”	Test whether unsupported questions are rejected.	The router returned a 400 Bad Request with the message that the request should be about demand or quantity.	Shows that the LLM-based service can enforce task boundaries instead of attempting to answer unsupported questions.

The main value of the LLM-based AI service is that it improves usability. A business user does not need to remember endpoint names, payload structure, or field names such as `material_name`, `month`, `year`, and `net_price`. Instead, the user can ask a natural-language question and the system translates that request into the correct service call. This lowers the barrier to use and makes the AI service more accessible to nontechnical users.

Another important value is orchestration. The LLM acts as an interface layer between people and enterprise services. That means it can sit in front of multiple APIs and route requests based on intent. Even in this assignment, the router demonstrates the basic enterprise pattern: classify the request, extract structured data, call the relevant service, and return the result in a user-friendly way.

At the same time, several challenges need to be addressed. Prompt interpretation can be ambiguous, especially if users omit information such as date, material, or price. The extraction layer can also fail if the prompt is vague or if the LLM returns malformed JSON. Another challenge is governance and security: the service needs secure key handling, consistent error management, and monitoring. In addition, the system should avoid hallucinating unsupported answers, which is why the invalid-prompt behavior is important. In a production setting, prompt validation, logging, fallback rules, and tighter enterprise controls would be necessary.

Conclusion

Overall, this assignment demonstrated the full AI-as-a-Service lifecycle: data extraction from Snowflake, demand model development, deployment of manual and AutoML services, and integration of an LLM-based interface on top of those services. The prototype shows how prediction models become more valuable when they are deployed as callable services and then wrapped in interfaces that make them easier to use in real business processes.